МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Системный анализ

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка многопользовательской блог-платформы с интеллектуальным помощником»

Выполнила:
Студентка 3 курса группы Д-Э317
Преснякова Вероника Борисовна

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

_____
учёная степень, учёное звание, ФИО

Члены комиссии:

_____ _____
учёная степень, учёное звание, ФИО подпись

_____ _____
учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение **3**

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АРХИТЕКТУРНЫЕ ПАТТЕРНЫ ВЕБ-ПРИЛОЖЕНИЙ **6**

1.1. Архитектурные паттерны и организация приложения . . . 6

1.2. Обоснование технологического стека 10

Глава 2. ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ **13**

2.1. Математическая модель игровой экономики и интеллектуальный помощник 13

2.2. Проектирование реляционной схемы базы данных 14

2.3. Программная реализация серверной и клиентской логики 16

2.4. Обеспечение безопасности и параллельного выполнения . 19

2.5. Стратегия тестирования и контроль качества 20

Заключение **23**

Список использованных источников **25**

Введение

В современных условиях экспоненциального роста объемов цифровой информации и возрастающей когнитивной нагрузки эффективное управление персональными знаниями становится критически важным фактором личной и профессиональной продуктивности. Ведение регулярных структурированных заметок позволяет пользователям фиксировать идеи, формировать базы знаний и снижать уровень информационной тревожности. Однако традиционные инструменты управления заметками и блог-платформы сталкиваются с фундаментальной проблемой долгосрочного удержания (retention): большинство пользователей забрасывают регулярное ведение записей уже в первые недели использования из-за отсутствия мгновенной обратной связи, однообразия рутинного процесса и необходимости постоянного волевого усилия.

Современные исследования в области проектирования пользовательского опыта показывают, что сухая самодисциплина значительно уступает механизмам эмоциональной вовлеченности и фоновой геймификации. Внедрение виртуального питомца, состояние и показатели благополучия которого органично зависят от регулярности и качества ведения заметок, формирует устойчивый паттерн возврата в приложение. При этом критически важно, чтобы игровые элементы функционировали в «теневом» режиме, не отвлекая пользователя всплывающими окнами и визуальным шумом в процессе интеллектуальной работы с текстом.

Дополнительным фактором, определяющим эффективность современной платформы управления контентом, является наличие интеллектуального помощника. По мере роста объема накопленных материалов ручная категоризация становится трудоемкой, а поиск информации затрудняется. Применение алгоритмов семантического автотегирования и кластеризации позволяет автоматизировать систематизацию базы знаний, преобразуя хаотичный набор заметок в упорядоченную структуру.

Создание высоконагруженных многопользовательских систем такого типа предъявляет повышенные требования к архитектурной надежности, масштабируемости и сквозной типизации данных. Необходимость параллельной обработки пользовательских HTTP-запросов и тяжелых фоновых расчетов игровой экономики (пересчета уровней, агрегации опыта, начисления наград) требует бескомпромиссного разделения ответственности между компонентами, эффективного управления конкурентным доступом на уровне СУБД и модульной структуры клиентского кода.

Актуальность темы обусловлена необходимостью разработки производительной, надежной и масштабируемой блог-платформы для ведения заметок, сочетающей строгий инструмент для работы с контентом, адаптивную модель фоновой геймификации для повышения мотивации пользователей и интеллек-

туального помощника для автоматической смысловой обработки данных.

Целью курсового проекта является проектирование и разработка многопользовательской блог-платформы (веб-приложения заметок «PigKeep») с интеллектуальным помощником и виртуальным питомцем, обладающей высокой степенью отказоустойчивости, строгой сквозной типизацией и оптимизированной архитектурой данных.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Исследовать теоретические основы построения многоуровневых клиент-серверных архитектур, паттернов проектирования (SoC, Repository Pattern, Event Log) и методологии Feature-Sliced Design (FSD) для модульной организации фронтенда.

2. Провести сравнительный анализ и обосновать выбор современного технологического стека, обеспечивающего сквозную типобезопасность (TypeScript, PostgreSQL 18, Drizzle ORM, Express 5, React 19, Vite, TanStack Query, Zustand, Zod, Tailwind CSS).

3. Разработать математическую модель игровой экономики приложения на основе аппарата многомерной взвешенной средней (расчет Индекса удовольствия, шкалы HP и системы коэффициентов опыта XP) и концепцию интеллектуального помощника.

4. Спроектировать реляционную схему базы данных в третьей нормальной форме (3NF), разработать структуру связей, составные покрывающие индексы и механизмы полнотекстового поиска с ранжированием релевантности.

5. Реализовать серверную часть (валидацию на уровне middleware, агрегацию прогресса с помощью обобщенных табличных выражений CTE, транзакционные рекомендательные блокировки `pg_advisory_xact_lock`) и клиентский интерфейс с поддержкой оптимистичных обновлений.

6. Разработать комплексную стратегию тестирования по паттерну AAA (модульные, интеграционные и контрактные тесты с изоляцией состояния) и реализовать процесс безопасного развертывания системы в контейнерах Docker.

Объектом исследования выступают веб-приложения для управления персональными заметками и блог-платформы, интегрированные с механизмами игровой экономики и интеллектуальной поддержки пользователей.

Предметом исследования являются программно-архитектурные решения, структуры данных реляционной СУБД, алгоритмы вычисления

многомерных показателей продуктивности и механизмы синхронизации состояния клиент-серверной системы.

Научная новизна работы заключается в синтезе метода многомерной взвешенной средней с архитектурным паттерном Event Log и транзакционными рекомендательными блокировками PostgreSQL, что позволило реализовать неинтрузивную фоновую игровую модель, гарантирующую консистентность состояния и защиту от состояний гонки без снижения пропускной способности системы.

Практическая значимость результатов заключается в создании полнофункционального веб-приложения PigKeep, обладающего готовым к развертыванию контейнеризированным монорепозиторием, компонентной базой на основе методологии FSD, оптимизированным ядром базы данных и набором автоматизированных тестов, обеспечивающих высокую надежность и удобство дальнейшего масштабирования продукта.

Структура работы: курсовой проект состоит из введения, двух глав, заключения и списка использованных источников.

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АРХИТЕКТУРНЫЕ ПАТТЕРНЫ ВЕБ-ПРИЛОЖЕНИЙ

1.1. Архитектурные паттерны и организация приложения

Современная инженерия программного обеспечения, особенно в контексте высоконагруженных клиент-серверных систем, требует бескомпромиссного соблюдения принципа разделения ответственности (Separation of Concerns, SoC). Монолитные структуры, в которых логика обработки сетевых запросов тесно переплетена с бизнес-правилами и SQL-запросами, неизбежно приводят к деградации кодовой базы (так называемому «спагетти-коду») и невозможности эффективного масштабирования. В основе серверной части проекта лежит строгая многоуровневая архитектура (Layered Architecture), которая обеспечивает жесткую изоляцию компонентов системы в зависимости от их семантической роли [1].

В рамках платформы Node.js жизненный цикл любого HTTP-запроса проходит через строго детерминированную цепочку уровней абстракции. Изначально запрос попадает в маршрутизатор (Router), задача которого сводится исключительно к определению конечной точки. Далее запрос проходит через слой промежуточных обработчиков (Middleware), где осуществляется авторизация и валидация полезной нагрузки. После этого управление передается контроллеру, выступающему в роли оркестратора бизнес-процессов. Однако контроллер не взаимодействует с базой данных напрямую; для абстрагирования слоя персистентности применяется классический паттерн «Репозиторий» (Repository Pattern) [1].

Паттерн «Репозиторий» выступает в качестве посредника между логикой предметной области и уровнем сопоставления данных, используя интерфейсы, аналогичные коллекциям объектов [4]. Применение репозиториев позволяет скрыть сложность построения SQL-запросов и детали реализации конкретной системы управления базами данных (СУБД) от бизнес-логики. Это архитектурное решение не только повышает читаемость кода, но и является критически важным для стратегии тестирования. Благодаря инверсии зависимостей (Dependency Inversion), реальный репозиторий, обращающийся к PostgreSQL, может быть легко заменен на mock-объект в модульных тестах, что позволяет проверять бизнес-логику в абсолютной изоляции от внешнего ввода-вывода [5].

```
interface RepositoryType {
  repo: {
    getNotesFromDB: (params?: {
      search?: string | undefined
      tagId?: string | undefined
      isArchive?: boolean | undefined
      isDeleted?: boolean | undefined
    }) => Promise<Note[]>
    getNoteByIdFromDB: (noteId: string) => Promise<Note | undefined>
```

```
    createNoteFromDB: (noteData: NoteCreateIn) => Promise<Note | undefined>
    putNoteFromDB: (noteId: string, noteData: NotePutIn) => Promise<Note | undefined
    >
    patchNoteFromDB: (noteId: string, noteData: NotePatchIn) => Promise<Note |
    undefined>
    deleteNoteFromDB: (noteId: string) => Promise<number | null>
    getNoteTagsFromDB: (noteId: string) => Promise<{ id: string; name: string }[]>
    addTagToNoteInDB: (noteId: string, tagId: string) => Promise<{ note_id: string;
    tag_id: string }[]>
    removeTagFromNoteInDB: (noteId: string, tagId: string) => Promise<number | null>
  }
}

export function controller({ repo }: RepositoryType) {
  const {
    getNotesFromDB,
    getNoteByIdFromDB,
    createNoteFromDB,
    putNoteFromDB,
    patchNoteFromDB,
    deleteNoteFromDB,
    getNoteTagsFromDB,
    addTagToNoteInDB,
    removeTagFromNoteInDB,
  } = repo;
}
```

Листинг 1.1: Определение интерфейса репозитория заметок и контроллера

На клиентской стороне сложность управления состоянием, компонентами и их взаимосвязями диктует необходимость использования продвинутых методологий организации кодовой базы. Традиционный подход группировки файлов по их техническому типу (папки `components`, `hooks`, `services`) быстро теряет свою эффективность по мере роста проекта, так как не отражает бизнес-домен приложения [6]. В связи с этим для фронтенда была выбрана архитектурная методология Feature-Sliced Design (FSD) [1]. В отличие от классических подходов, FSD группирует код на основе логики предметной области, решая фундаментальную проблему запутанных зависимостей [8].

Методология Feature-Sliced Design вводит строгую иерархию слоев (Layers), направленную от глобального к локальному:

- **App**: Слой инициализации приложения. Здесь располагаются корневые настройки, глобальные стили, базовые провайдеры контекста и конфигурация маршрутизации [8].
- **Pages**: Слой страниц. Модули этого слоя собирают полноценные пользовательские экраны путем композиции сущностей, фич и виджетов [9].
- **Widgets**: Самостоятельные, крупные композиционные блоки интерфейса, такие как панель навигации или сложный дашборд управления питомцем [9].
- **Features**: Слой пользовательских сценариев, несущих конкретную бизнес-ценность (например, функционал «создать заметку», «покормить

питомца», «добавить тег»). Фичи инкапсулируют логику взаимодействия пользователя с системой [9].

- **Entities**: Слой бизнес-сущностей. Содержит интерфейсы данных, UI-карточки и логику, напрямую связанную с объектами предметной области (пользователь, заметка, питомец), но без привязки к конкретным действиям [9].

- **Shared**: Слой переиспользуемых инфраструктурных модулей. Включает базовые UI-компоненты (кнопки, инпуты), утилиты, конфигурации API-клиентов и абстракции, не зависящие от бизнес-логики [10].

Краеугольным камнем FSD является правило однонаправленного потока зависимостей: модуль имеет право импортировать только те модули, которые располагаются на строго нижестоящих слоях [10]. Это архитектурное ограничение гарантирует, что изменения в высокоуровневых компонентах (например, на страницах) никогда не приведут к каскадным поломкам в базовых сущностях. Кроме того, FSD внедряет концепцию Public API: каждый модуль (слайс) экспортирует свою функциональность исключительно через единый файл `index.ts`. Все внутренние файлы модуля остаются инкапсулированными и недоступными для прямого импорта извне, что защищает систему от неконтролируемого связывания (tight coupling) [7]. На рисунке 1.1 приведена иерархия слоев FSD.

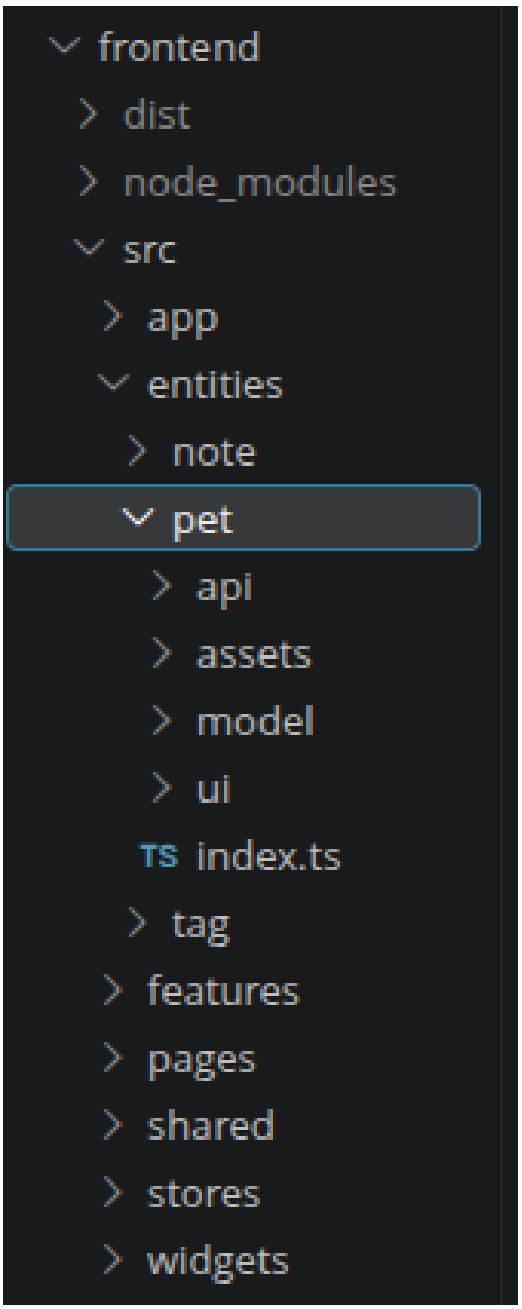


Рис. 1.1. Иерархия слоев методологии Feature-Sliced Design

Геймификационная составляющая сервиса, отвечающая за развитие питомца, опирается на архитектурный паттерн Event Log, который идеологически близок к концепции Event Sourcing. В традиционных системах CRUD любое действие пользователя (например, создание заметки) привело бы к мгновенному изменению состояния питомца (увеличению опыта в таблице `pets`). Однако такой подход уязвим для состояний гонки и затрудняет аудит [1]. Вместо мутации состояния, паттерн Event Log фиксирует действия пользователя как серию атомарных, неизменяемых событий в специализированной таблице. Эти факты в дальнейшем асинхронно обрабатываются фоновым планировщиком с использованием транзакционных рекомендательных блокировок (Advisory Locks), что позволяет реализовать сложную математическую логику без блокировки основного потока выполнения пользовательских запросов.

1.2. Обоснование технологического стека

Выбор технологического стека является одним из наиболее критичных этапов проектирования, поскольку он определяет пределы масштабируемости, производительности и надежности системы. В проекте стек технологий продиктован строгими требованиями к сквозной типизации данных (End-to-End Type Safety) и отказоустойчивости [1].

В качестве системы управления реляционными базами данных выбрана PostgreSQL 18 [1]. Выбор в пользу PostgreSQL, в противовес легковесной синхронной SQLite или документо-ориентированной MongoDB, обусловлен несколькими архитектурными преимуществами. Во-первых, природа данных (заметки, теги, связи между ними) носит ярко выраженный реляционный характер, требующий соблюдения ограничений внешних ключей (Foreign Key Constraints). Во-вторых, PostgreSQL реализует архитектуру Multi-Version Concurrency Control (MVCC) — многоверсионное управление конкурентным доступом [12].

В традиционных СУБД читающие транзакции блокируют пишущие, и наоборот. MVCC элегантно решает эту проблему: каждая транзакция работает с собственным консистентным «снимком» (snapshot) данных на момент своего начала [12]. Технически это реализуется за счет скрытых системных колонок `xmin` (идентификатор транзакции, создавшей версию строки) и `xmax` (идентификатор транзакции, удалившей или обновившей строку) [12]. Благодаря этому, длительные фоновые расчеты опыта питомца могут выполняться без ущерба для пользователей, которые в этот же момент интенсивно создают или читают свои заметки. PostgreSQL также предоставляет мощнейшие инструменты полнотекстового поиска на уровне ядра СУБД, что критически важно для приложения заметок.

Для интеграции серверной логики с СУБД выбран инструмент Drizzle ORM [1]. В экосистеме TypeScript исторически доминировал инструмент Prisma, однако в контексте архитектуры данного проекта Drizzle ORM предоставляет неоспоримые преимущества. Prisma использует парадигму «Schema-first»: разработчик описывает структуру базы данных в специализированном файле `.prisma`, после чего запускает процесс кодогенерации, который создает типизированный клиент [16]. Несмотря на удобство абстракции, Prisma исторически опиралась на тяжеловесный runtime-движок (Query Engine), что приводило к значительному расходу оперативной памяти и проблемам с задержками при «холодном старте» (cold start) в serverless/edge окружениях [17].

Drizzle ORM, напротив, исповедует принцип «SQL-first» и «Codebase-first». Описание схемы базы данных выполняется на языке TypeScript [16]. Отсутствие этапа кодогенерации означает, что типы выводятся мгновенно в процессе написания кода (Type Inference). Главное преимущество Drizzle — нулевые накладные расходы на runtime. Drizzle напрямую транслирует вызовы

функций в оптимизированные SQL-строки без промежуточного слоя движка, что сокращает накладные расходы на формирование запроса до ~ 0.5 мс [17]. Подробное сравнение подходов приведено в таблице 1.1.

Таблица 1.1. Сравнительный анализ Drizzle ORM и Prisma ORM

Архитектурный аспект	**Drizzle ORM**	**Prisma ORM**
Парадигма проектирования	SQL-first, Codebase-first [16]	Schema-first, абстракция объектов [16]
Накладные расходы (Overhead)	Минимальные, прямая генерация SQL-строк [17]	Наличие Query Compiler (WASM в Prisma 7) [17]
Система типизации	Мгновенный вывод (Inference) без кодогенерации [16]	Строгая типизация через `prisma generate` [16]
Выполнение в Edge (Serverless)	Нативная поддержка, нулевые зависимости [16]	Улучшено в v7, но требует дополнительных адаптеров [17]
Подход к миграциям	Прозрачные, редактируемые SQL-файлы [16]	Скрыты за абстрактным DSL [16]

В листинге 1.2 представлен фрагмент описания сущностей `pets`, `notes` и `tags` с использованием синтаксиса Drizzle ORM.

```
export const pets = pgTable('pets', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  name: varchar('name', { length: 255 }).notNull(),
  level: integer('level').default(1).notNull(),
  progress: integer('progress').default(0).notNull(),
  pleasureIndex: integer('pleasure_index').default(100).notNull(),
})

export const notesTable = pgTable('notes', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  title: text().default('').notNull(),
  content: text().default('').notNull(),
  isArchive: boolean().default(false),
  isDeleted: boolean().default(false),
})

export const tagsTable = pgTable('tags', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  name: text().notNull(),
})
```

Листинг 1.2: Описание схемы сущностей pets, notes и tags в Drizzle ORM

Серверная инфраструктура базируется на платформе Node.js и веб-фреймворке Express 5. Использование Node.js позволяет унифицировать язык программирования (TypeScript) на всем стеке, обеспечивая возможность переиспользования интерфейсов данных между клиентом и сервером.

Express 5 привносит нативную поддержку асинхронных операций (Promises) в обработчиках маршрутов, избавляя разработчиков от необходимости оборачивать каждый контроллер в громоздкие блоки `try/catch` или использовать сторонние библиотеки вроде `express-async-handler` [1].

На стороне клиентского приложения архитектура состояния разделена на две независимые области. Фреймворк React 19 в связке со сборщиком Vite обеспечивает компонентную модель и феноменально быструю горячую замену модулей (HMR). Для управления серверным состоянием (Server State) — данными, которые асинхронно извлекаются из API и требуют кэширования, дедупликации запросов и фоновой синхронизации — внедрена библиотека TanStack Query v5 [1]. Управление локальным пользовательским состоянием, таким как состояние модальных окон, тема оформления или временные фильтры, делегировано библиотеке Zustand. В отличие от React Context или Redux, Zustand предоставляет минималистичный API, основанный на хуках, и точечно обновляет компоненты, предотвращая избыточный рендеринг всего дерева [1].

Сквозная валидация данных реализуется с помощью библиотеки Zod, позволяющей единожды описать схему данных и использовать ее как для валидации на сервере, так и для форм на клиенте. Визуальная составляющая формируется посредством Tailwind CSS v4, который, используя utility-first подход, радикально уменьшает объем генерируемого CSS-кода за счет атомарных классов [1].

Глава 2. ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1. Математическая модель игровой экономики и интеллектуальный помощник

Сердцем платформы, отличающим его от конкурентов, является сложная математическая модель игровой экономики. Основная бизнес-цель данной модели — создание устойчивого паттерна возврата (retention) и долгосрочного удержания пользователя через психологию привязанности к виртуальному питомцу [1].

Центральной метрикой системы является Индекс удовольствия (Health/ Happiness Points, HP), неформально именуемый «Свинометром». Для вычисления этого индекса применяется алгоритм многомерной средней. Модель не просто суммирует количество написанных символов, а оценивает комплексную полезность действий пользователя через систему весов. В расчет берутся такие векторы, как регулярность входов в приложение (streak), объем и структурированность заметок, а также частота использования тегов (полезная организация информации).

Математический расчет осуществляется в несколько этапов. Сначала каждый j-й показатель активности пользователя x_j за отчетные сутки нормируется относительно установленного целевого эталона (дневной нормы) X_j^{norm}:

$$z_j = \min\left(1, \frac{x_j}{X_j^{\text{norm}}}\right), \tag{2.1}$$

где x_j — фактическое число действий j-го типа за расчетный период; X_j^{norm} — эталонная суточная норма активности; $z_j \in [0; 1]$ — коэффициент выполнения нормы.

Интегральный синтетический показатель настроения и благополучия питомца вычисляется по формуле многомерной взвешенной средней:

$$\bar{I} = 100 \times \frac{\sum_{j=1}^{m} w_j \cdot z_j}{\sum_{j=1}^{m} w_j}, \tag{2.2}$$

где m — количество учитываемых типов активности ($m = 4$); w_j — весовые коэффициенты значимости действий. При создании новой заметки вес составляет $w_1 = 20$ (норма $X_1^{\text{norm}} = 3$); при категоризации тегами $w_2 = 15$ (норма $X_2^{\text{norm}} = 4$); при редактировании и обновлении $w_3 = 10$ (норма $X_3^{\text{norm}} = 5$); при ежедневном входе в приложение $w_4 = 5$ (норма $X_4^{\text{norm}} = 1$). Полученный суточный индекс $\bar{I} \in [0; 100]$ определяет шкалу здоровья и настроения питомца.

Полученный индекс HP выступает в роли динамического множителя для накопления опыта (XP). Накопление опыта позволяет питомцу визуально эволюционировать. Если индекс HP находится на высоком уровне (что свидетельствует об активном и качественном ведении заметок), к получаемому базовому опыту применяется стимулирующий коэффициент $\times 1.2$. В случае длительного бездействия пользователя алгоритм фиксирует падение Индекса удовольствия, что приводит к активации штрафного коэффициента $\times 0.5$. Подобная механика создает эффект FOMO (Fear Of Missing Out — страх упущенной выгоды). Для предотвращения злоупотреблений и намеренного флуда бессмысленными записями в систему внедрен жесткий лимит — пользователь не может получить более 500 XP за одни календарные сутки [1].

Для практической реализации заявленной темы курсового проекта в архитектуру сервиса закладывается интеллектуальный помощник (AI Smart Assistant). Ассистент персонифицирован в лице питомца Бориса и выполняет следующие задачи:

1. **Семантическое автотегирование в реальном времени**: в момент сохранения заметки фоновый алгоритм анализирует текст заголовка и тела, извлекая ключевые тематические сущности и предлагая 2–3 наиболее релевантных тега в один клик.

2. **Смысловая кластеризация заметок**: периодический анализ векторных представлений (эмбеддингов) текстов неструктурированных заметок выявляет скрытые тематические группы и предлагает объединить разрозненные мысли в единую категорию.

3. **Геймифицированная интеграция**: принятие рекомендаций интеллектуального помощника рассматривается системой как операция структурирования данных, поощряя пользователя дополнительным приростом опыта и повышением Индекса удовольствия питомца.

2.2. Проектирование реляционной схемы базы данных

Проектирование реляционной схемы базы данных выполнено в строгом соответствии с правилами третьей нормальной формы (3NF). Это архитектурное решение исключает дублирование данных и защищает систему от аномалий модификации (update anomalies). Основные сущности системы образуют следующую структуру:

- Таблица `notes` (заметки). Содержит первичный ключ (UUID), заголовок, текстовое содержимое и временные метки создания/изменения.

- Таблица `tags` (теги). Инкапсулирует название тега.

- Таблица `noteTags` (связывающая таблица). Необходима для физической реализации связи «многие ко многим», позволяя одной заметке иметь множество тегов, а одному тегу относиться к множеству заметок.
- Таблица `action_logs`. Является реализацией паттерна Event Log, храня иммутабельные записи о каждом действии пользователя для последующего асинхронного начисления XP [1].

В листинге 2.1 представлена программная реализация связующей таблицы и отношений многие-ко-многим с использованием Drizzle ORM.

```
export const noteTagsTable = pgTable(
  'noteTags',
  {
    note_id: uuid('note_id')
      .notNull()
      .references(() => notesTable.id, { onDelete: 'cascade' }),
    tag_id: uuid('tag_id')
      .notNull()
      .references(() => tagsTable.id, { onDelete: 'cascade' }),
  },
  (table) => [primaryKey({ columns: [table.tag_id, table.note_id] })]
)

export const noteTagsRelations = defineRelations({ tagsTable, notesTable,
    noteTagsTable }, (r) => ({
  tagsTable: {
    notesTable: r.many.notesTable({
      from: r.tagsTable.id.through(r.noteTagsTable.tag_id),
      to: r.notesTable.id.through(r.noteTagsTable.note_id),
    }),
  },
  notesTable: {
    tagsTable: r.many.tagsTable({
      from: r.notesTable.id.through(r.noteTagsTable.note_id),
      to: r.tagsTable.id.through(r.noteTagsTable.tag_id),
    }),
  },
}))
```

Листинг 2.1: Реализация связи многие-ко-многим между заметками и тегами в Drizzle ORM

В листинге 2.2 приведено описание таблицы журнала действий `action_logs` и ее индексов.

```
export const actionLogs = pgTable(
  'action_logs',
  {
    id: uuid('id')
      .primaryKey()
      .default(sql`gen_random_uuid()`),
    petId: uuid('pet_id')
      .notNull()
      .references(() => pets.id, { onDelete: 'cascade' }),
    actionTypeId: integer('action_type_id')
      .notNull()
      .references(() => actionTypes.id, { onDelete: 'restrict' }),
    createdAt: timestamp('created_at', { withTimezone: true })
```

```
      .defaultNow()
      .notNull(),
  },
  (table) => [
    index('action_logs_pet_id_idx').on(table.petId),
    index('action_logs_created_at_idx').on(table.createdAt),
    index('idx_action_logs_cron_covering').on(
      table.createdAt,
      table.petId,
      table.actionTypeId
    ),
  ]
)
```

Листинг 2.2: Описание таблицы журнала событий action_logs и индексов

Для обеспечения мгновенного поиска по накопленной базе знаний приложение не использует стандартный оператор LIKE, который крайне неэффективен при поиске по большим объемам текста из-за невозможности использования B-Tree индексов при поиске по подстроке. Вместо этого задействован встроенный механизм полнотекстового поиска PostgreSQL. Текстовые данные преобразуются с помощью функции `to_tsvector` в оптимизированный вектор лексем (с удалением стоп-слов и приведением к корню слова). Поисковые запросы пользователя обрабатываются через `to_tsquery`. Использование функции `ts_rank` позволяет базе данных самостоятельно ранжировать возвращаемые результаты на основе плотности и расположения искомых слов в заметке, обеспечивая высокую релевантность выдачи [1].

Производительность фоновых задач (например, ежедневного пересчета агрегированного прогресса) критически зависит от индексов. В базе данных спроектированы составные покрывающие индексы. Они сконструированы таким образом, что содержат не только ключи для фильтрации (например, `user_id`, `created_at`), но и сами агрегируемые значения. Это позволяет планировщику PostgreSQL выполнять запросы в режиме Index-Only Scan. В этом режиме данные извлекаются исключительно из структуры индекса, расположенной в оперативной памяти, без необходимости ресурсоемкого чтения основных страниц таблицы (heap) с диска, что обеспечивает масштабируемость системы [1].

2.3. Программная реализация серверной и клиентской логики

Практическая реализация серверной логики фокусируется на обеспечении отказоустойчивости и глубокой защите (Defense in Depth). Централизованная валидация данных реализована с использованием паттерна Middleware. Любой входящий HTTP-запрос (например, создание заметки) перехватывается промежуточным слоем, где его полезная нагрузка (body) сопоставляется со строгими схемами, описанными на базе Zod. Если данные не соответствуют контракту (например, отсутствует заголовок или текст превышает допустимый лимит), Middleware мгновенно прерывает дальнейшую обработку и возвращает

клиенту HTTP статус 400 (Bad Request). Это гарантирует, что в контроллеры и репозитории попадают исключительно очищенные и корректные данные, предотвращая инъекции и сбои в бизнес-логике [1].

Одной из самых сложных алгоритмических задач в бэкенде является реализация функции `processDailyPetProgress`. Поскольку расчет дневного прогресса зависит от сотен записей в таблице `action_logs`, построчное извлечение и обновление каждой записи на уровне Node.js неизбежно привело бы к проблеме N+1 запросов и переполнению пула соединений. Для оптимизации этой задачи используются Common Table Expressions (CTE) — обобщенные табличные выражения, реализуемые через оператор WITH в SQL. Использование CTE позволяет базе данных предварительно агрегировать события и вычислить итоговые инкременты опыта, а затем обновить таблицу питомцев одним атомарным запросом (Bulk Update) на уровне ядра СУБД [1]. В листинге 2.3 представлен соответствующий фрагмент SQL-запроса с CTE.

```
const recalculationQuery = sql<{
  id: string
  old_level: number
  new_level: number
  new_progress: number
}>`
  WITH daily_xp AS (
    SELECT
      ${actionLogs.petId} AS pet_id,
      SUM(${actionTypes.weight})::int AS earned_xp
    FROM ${actionLogs}
    INNER JOIN ${actionTypes}
      ON ${actionLogs.actionTypeId} = ${actionTypes.id}
    WHERE
      ${actionLogs.createdAt} >= ${startDate}
      AND ${actionLogs.createdAt} < ${endDate}
    GROUP BY ${actionLogs.petId}
  )
  UPDATE ${pets} AS p
  SET
    level = CASE
      WHEN lr.required_xp IS NOT NULL
           AND (p.progress + dxp.earned_xp) >= lr.required_xp
        THEN p.level + 1
      ELSE p.level
    END,
    progress = CASE
      WHEN lr.required_xp IS NOT NULL
           AND (p.progress + dxp.earned_xp) >= lr.required_xp
        THEN (p.progress + dxp.earned_xp) - lr.required_xp
      ELSE p.progress + dxp.earned_xp
    END
  FROM daily_xp dxp
  LEFT JOIN ${levelRequirements} lr
    ON lr.level = p.level
  WHERE p.id = dxp.pet_id
  RETURNING
    p.id,
    (p.level - CASE
      WHEN lr.required_xp IS NOT NULL AND (p.progress + dxp.earned_xp) >= lr.
    required_xp THEN 1
      ELSE 0
```

```
    END) AS old_level,
    p.level AS new_level,
    p.progress AS new_progress;
`
```

Листинг 2.3: Агрегация опыта и пересчет уровня питомца с использованием CTE

Интерфейс пользователя спроектирован с учетом высоких требований к отзывчивости (UX). Для нивелирования сетевых задержек реализован паттерн оптимистичных обновлений средствами TanStack Query v5 [1]. При добавлении тега или заметки интерфейс не дожидается ответа от сервера (HTTP 201 Created). В функции `onMutate` система делает «снимок» (snapshot) текущего состояния кэша, после чего мгновенно обновляет локальные данные, перерисовывая интерфейс с нулевой задержкой [19]. Фоном запрос отправляется на сервер. Если сервер возвращает ошибку (например, потеря соединения), срабатывает механизм отката (rollback) в блоке `onError`, который восстанавливает состояние интерфейса до ранее сохраненного снимка. Контроль форм и их реактивность осуществляются библиотекой `react-hook-form`, которая бесшовно интегрирована с Zod-схемами [1].

На рисунке 2.1 представлен интерфейс главного дашборда приложения PigKeep, объединяющий сетку заметок и панель виртуального питомца.

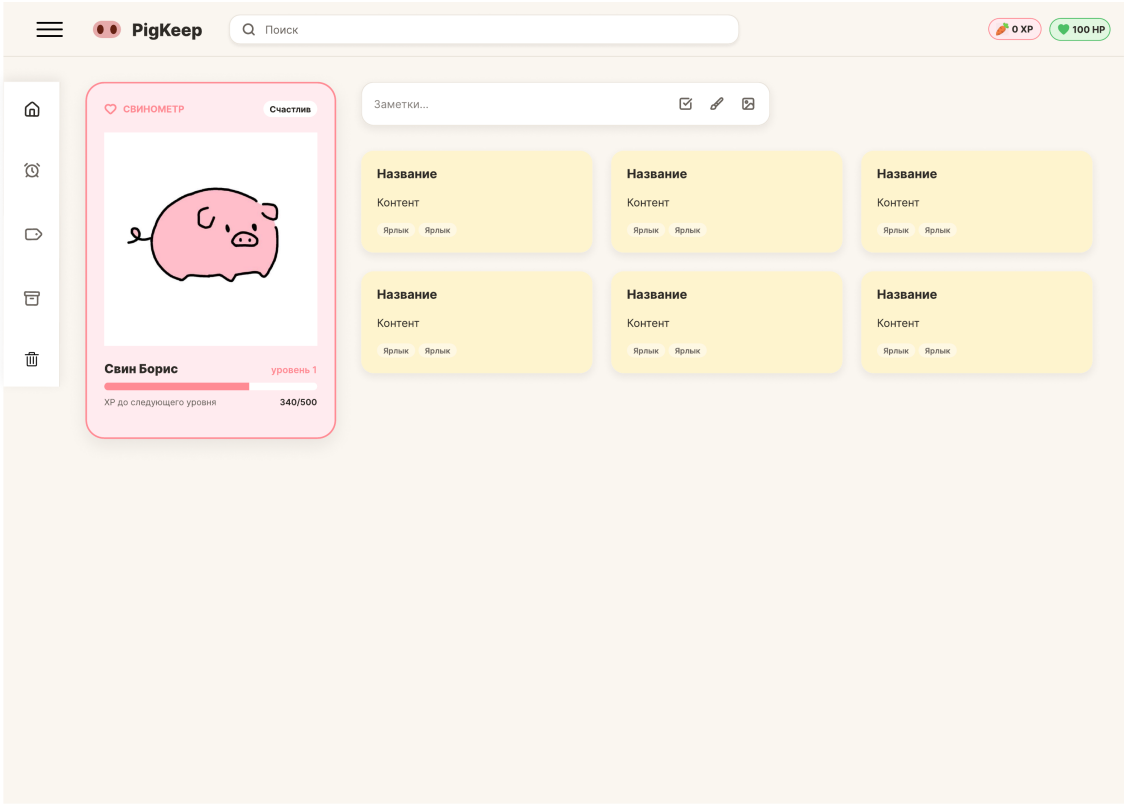


Рис. 2.1. Интерфейс главного экрана веб-приложения PigKeep

2.4. Обеспечение безопасности и параллельного выполнения

Безопасность параллельного выполнения фоновых задач обеспечивается через транзакционные рекомендательные блокировки (Advisory Locks) PostgreSQL. Если система развернута в нескольких экземплярах (например, в Kubernetes), существует риск, что несколько контейнеров попытаются выполнить ежедневный пересчет одновременно, что приведет к дублированию начисления опыта [11]. Использование блокировок строк (SELECT ... FOR UPDATE) в данном случае не подходит, так как физические строки могут еще не существовать [18]. PostgreSQL предоставляет функцию `pg_advisory_xact_lock`, которая создает логическую блокировку на основе сгенерированного приложением 64-битного ключа [1]. В отличие от блокировок уровня сессии (`pg_advisory_lock`), транзакционные блокировки (`pg_advisory_xact_lock`) автоматически освобождаются при коммите или откате транзакции. Это исключает риск «зависших» блокировок, особенно при использовании пулеров соединений, таких как PgBouncer, где понятие сессии размыто [11].

Процесс развертывания системы стандартизирован посредством контейнеризации в Docker с применением Multi-stage сборок (многоэтапных сборок). Этот подход решает две задачи: минимизацию размера образа и повышение безопасности. На первом этапе (Builder) в контейнер устанавливаются все зависимости (включая devDependencies), выполняется проверка типов TypeScript и сборка React-приложения через Vite. На втором этапе создается чистый, легковесный образ на базе Nginx. В него копируются только скомпилированные артефакты из папки dist. Важнейшим аспектом безопасности является использование образа nginx-unprivileged, который запускает веб-сервер от имени ограниченного пользователя (не root). Благодаря такому подходу итоговый размер Docker-образа фронтенда составляет всего 26.5 МБ [1].

В листинге 2.4 приведен многоэтапный Dockerfile для сборки и развертывания серверной части приложения.

```
FROM node:24.21.0-bookworm-slim AS builder
WORKDIR /usr/src/app
COPY package*.json ./
COPY backend/package*.json ./backend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci
COPY packages/shared ./packages/shared
COPY backend ./backend
WORKDIR /usr/src/app/backend
RUN npm run build:prod

FROM node:24.21.0-bookworm-slim AS runner
WORKDIR /usr/src/app
COPY package*.json ./
COPY backend/package*.json ./backend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci --omit=dev
COPY --from=builder /usr/src/app/packages/shared ./packages/shared
COPY --from=builder /usr/src/app/backend/build ./backend/build
```

```
COPY --from=builder /usr/src/app/backend/migrations ./migrations
EXPOSE 3000
USER node
CMD [ "node", "backend/build/server.js"]
```

Листинг 2.4: Многоэтапный Dockerfile для серверной части приложения

В листинге 2.5 приведен Dockerfile для развертывания клиентской части с сервером Nginx от имени непривилегированного пользователя.

```
FROM node:24.21.0-bookworm-slim AS builder
WORKDIR /usr/src/app
COPY package*.json ./
COPY frontend/package*.json ./frontend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci
COPY packages/shared ./packages/shared
COPY frontend ./frontend
WORKDIR /usr/src/app/frontend
RUN npm run build

FROM nginxinc/nginx-unprivileged:alpine AS runner
COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /usr/src/app/frontend/dist /usr/share/nginx/html
EXPOSE 8080
USER nginx
CMD ["nginx", "-g", "daemon off;"]
```

Листинг 2.5: Многоэтапный Dockerfile для развертывания клиентской части

2.5. Стратегия тестирования и контроль качества

Обеспечение качества программного обеспечения базируется на комплексном тестировании. Модульное тестирование алгоритмов (таких как расчет Свинометра) и схем валидации выполняется в среде Vitest, которая обладает нативной поддержкой конфигураций Vite. Для проверки интеграции UI и логики запросов без развертывания реальной базы данных применяется Mock Service Worker (MSW). MSW перехватывает сетевые запросы на уровне Service Worker в браузере или Node.js, позволяя возвращать заранее подготовленные мок-ответы. Это дает возможность эффективно тестировать краевые случаи, например, гарантированно вызывать ошибки сервера (HTTP 500) для проверки корректности работы механизма оптимистичного отката в UI [1].

В проекте реализована комплексная пирамида тестирования, охватывающая как клиентскую, так и серверную части, с фокусом на проверке реального поведения системы. На уровне контрактов (Vitest, Zod) проверяются граничные условия без запуска сервера. Интеграционное тестирование бэкенда (Supertest, Drizzle ORM, PostgreSQL) проверяет сквозное прохождение HTTP-запросов, работу полнотекстового поиска и соответствие БД нормальным формам.

В листинге 2.6 представлен интеграционный тест эндпоинта инициализации питомца с использованием библиотеки Supertest.

```
describe('GET /api/v1/pets', () => {
```

```
  it('should auto-initialize and return default pet when no pet exists', async () =>
     {
    const res = await request(app).get(BASE_URL)

    expect(res.status).toBe(200)
    expect(res.body.pet).toBeDefined()
    expect(res.body.pet).toMatchObject({
      name: 'Борис',
      level: 1,
      progress: 0,
      pleasureIndex: 100,
      requiredXp: 500,
      mood: 'happy',
    })
    expect(res.body.pet.id).toBeDefined()
  })
})
```

Листинг 2.6: Интеграционный тест эндпоинта API питомца с использованием Supertest

В листинге 2.7 представлен модульный тест проверки колонок схемы базы данных на соответствие требованиям 3NF.

```
describe('Database Schemas (3NF)', () => {
  it('should define pets table with correct columns', () => {
    expect(pets.id).toBeDefined()
    expect(pets.name).toBeDefined()
    expect(pets.level).toBeDefined()
    expect(pets.progress).toBeDefined()
    expect(pets.pleasureIndex).toBeDefined()
  })
})
```

Листинг 2.7: Тестирование колонок схемы базы данных

На фронтенде интеграционное тестирование (React Testing Library, MSW) оценивает реальные пользовательские сценарии, состояния загрузки и обработку ошибок интерфейса. Юнит-тестирование хуков гарантирует корректную инвалидацию кэша (TanStack Query) и управление таймерами, что предотвращает утечки памяти при размонтировании компонентов. В листинге 2.8 приведен тест компонента отображения карточки питомца `PetCard`.

```
describe('PetCard', () => {
  const mockPet: Pet = {
    id: 'pet-123',
    name: 'Борис',
    level: 2,
    progress: 340,
    requiredXp: 500,
    pleasureIndex: 90,
    mood: 'happy',
  }

  it("should render pet name with Свин'' prefix, level, and XP progress", () => {
    renderWithProviders(<PetCard pet={mockPet} />)

    expect(screen.getAllByText('Свин Борис').length).toBeGreaterThan(0)
    expect(screen.getAllByText('уровень 2').length).toBeGreaterThan(0)
```

```
    expect(screen.getByText('340/500')).toBeInTheDocument()
    expect(screen.getByText('XP до следующего уровня')).toBeInTheDocument()
  })
})
```

Листинг 2.8: Тестирование компонента PetCard с использованием React Testing Library

Тесты строятся по паттерну AAA (Arrange-Act-Assert) и используют вспомогательные хелперы для подготовки данных, исключая дублирование кода. Изоляция тестов достигается за счет полной очистки выделенной БД перед каждым запуском, применения независимых клиентов для кэша и оборачивания компонентов в кастомные провайдеры контекста (листинг 2.9).

```
beforeEach(async () => {
  await test_db.execute(sql`TRUNCATE TABLE ${notesTable} CASCADE`)
})
```

Листинг 2.9: Изоляция тестовых наборов через очистку таблиц базы данных

Заключение

В ходе выполнения курсового проекта была спроектирована и разработана многопользовательская блог-платформа (веб-приложение заметок «PigKeep») с интеллектуальным помощником и виртуальным питомцем, ориентированная на решение проблемы долгосрочного удержания пользователей и стимулирование регулярной продуктивной работы.

В процессе выполнения работы были достигнуты следующие ключевые результаты:

1. Проведен углубленный анализ архитектурных паттернов организации современных распределенных веб-систем. Серверная часть реализована на основе многоуровневой архитектуры (Layered Architecture) с разделением слоев Router, Middleware, Controller и Repository. На клиенте применена архитектурная методология Feature-Sliced Design (FSD), что исключило неконтролируемые циклические зависимости и обеспечило изоляцию функциональных модулей (App, Pages, Widgets, Features, Entities, Shared).

2. Обоснован и внедрен технологический стек, гарантирующий сквозную типобезопасность (End-to-End Type Safety): платформа Node.js, веб-фреймворк Express 5, клиентская библиотека React 19 в связке с Vite, инструментарий управления серверным состоянием TanStack Query v5, легковесный менеджер локального состояния Zustand, валидатор схем Zod и утилитарный фреймворк Tailwind CSS v4.

3. В качестве системы персистентности выбрана СУБД PostgreSQL 18 и библиотека Drizzle ORM. Применение SQL-first подхода обеспечило нулевые runtime-издержки (~ 0.5 мс на формирование запроса) и автоматический вывод TypeScript-типов без этапа кодогенерации, показав существенные преимущества перед Prisma ORM.

4. Разработана и математически обоснована модель игровой экономики на базе аппарата многомерной взвешенной средней (Индекс удовольствия, или Свинометр). Алгоритм оценивает разнородные векторы пользовательской активности (регулярность сессий, создание и редактирование заметок, структурирование тегами), формируя динамические множители опыта ($\times 1.2$ и $\times 0.5$) и суточный лимит 500 XP для защиты от накрутки.

5. Спроектирована реляционная схема базы данных в третьей нормальной форме (3NF), исключающая аномалии обновления. Внедрены составные покрывающие индексы, обеспечивающие выполнение аналитических выборок в режиме Index-Only Scan, а также механизм полнотекстового поиска PostgreSQL с векторизацией лексем и ранжированием релевантности.

6. Разработана система асинхронного пересчета прогресса на базе Common Table Expressions (CTE), что устранило проблему $N + 1$ запросов и обеспечило атомарный Bulk Update на уровне ядра СУБД. Конкурентная безопасность параллельных фоновых процессов гарантирована транзакционными рекомендательными блокировками `pg_advisory_xact_lock`.

7. Реализован механизм оптимистичных обновлений интерфейса с функцией отката состояния при сбоях сети, что обеспечило мгновенный отклик интерфейса.

8. Построена пирамида автоматизированного тестирования по паттерну AAA (Arrange-Act-Assert), включающая контрактные тесты схем Zod, интеграционные тесты API с библиотекой Supertest, тестирование изолированной тестовой БД и тестирование фронтенд-компонентов с помощью React Testing Library и Mock Service Worker (MSW).

9. Стандартизирован процесс сборки и безопасного развертывания монорепозитория с помощью многоэтапных Dockerfile. Использование непривилегированного пользователя в образе `nginx-unprivileged` позволило получить легковесный контейнер фронтенда объемом всего 26.5 МБ.

Перспективы дальнейшего развития проекта включают интеграцию векторной базы данных pgvector для реализации глубокого семантического поиска по смысловой близости заметок, подключение локальных моделей больших языковых моделей (LLM) для автоматического реферирования и смысловой кластеризации контента, а также расширение функционала совместной работы пользователей в реальном времени посредством WebSockets.

Список использованных источников

[1] Architectural Patterns Revisited – A Pattern Language : веб-сайт. — URL: https://eprints.cs.univie.ac.at/2698/1/ArchPatterns.pdf (дата обращения: 05.10.2026).

[2] Does Domain-Driven Design Lead to Finding the Optimal Modularity : веб-сайт. — URL: http://ieeexplore.ieee.org/iel7/6287639/9312710/09359794.pdf (дата обращения: 05.10.2026).

[3] An Approach for Mapping Domain-Specific AOM Applications to a : веб-сайт. — URL: https://www.jucs.org/jucs_20_4/an_approach_for_mapping/jucs_20_04_0534_0560_matsumoto.pdf (дата обращения: 05.10.2026).

[4] A Pattern Language for Metadata-based Frameworks : веб-сайт. — URL: https://bia.unibz.it/view/pdfCoverPage?instCode=39UBZ_INST&filePid=13300047980001241&download=true (дата обращения: 05.10.2026).

[5] Feature Sliced Design: The Smart Way To Scale Fast : веб-сайт. — URL: https://denebrixai.com/blog/feature-sliced-design-guide/ (дата обращения: 05.10.2026).

[6] Feature-Sliced Design: Welcome : веб-сайт. — URL: https://feature-sliced.design/ (дата обращения: 05.10.2026).

[7] Feature-Sliced Design: A Guide To Scalable Frontend Architecture : веб-сайт. — URL: https://www.godeltech.com/blog/feature-sliced-design-a-guide-to-scalable-frontend-architecture/ (дата обращения: 05.10.2026).

[8] Feature-Sliced Design and good frontend architecture : веб-сайт. — URL: https://www.codecentric.de/en/knowledge-hub/blog/feature-sliced-design-and-good-frontend-architecture (дата обращения: 05.10.2026).

[9] Feature-Sliced Design: The Ideal Frontend Architecture : веб-сайт. — URL: https://medium.com/@dtgasparyan/feature-sliced-design-the-ideal-frontend-architecture-84d701ad44ba (дата обращения: 05.10.2026).

[10] Feature Sliced Design: Frontend Architecture Guide : веб-сайт. — URL: https://physicsfundamentalsguide.com/feature-sliced-design-frontend-guide/ (дата обращения: 05.10.2026).

[11] Postgres Concurrency Internals: Connections, Locks, and MVCC : веб-сайт. — URL: https://abhimanyunagurkar.com/blog/postgres-concurrency-internals/ (дата обращения: 05.10.2026).

[12] MVCC in PostgreSQL: Benefits and Challenges : веб-сайт. — URL: https://www.scribd.com/document/835805809/Multi-Version-Concurrency-Control-MVCC-in-PostgreSQL (дата обращения: 05.10.2026).

[13] Multiversion Concurrency Control (MVCC) in PostgreSQL : веб-сайт. — URL: https://www.geeksforgeeks.org/postgresql/multiversion-concurrency-control-mvcc-in-postgresql/ (дата обращения: 05.10.2026).

[14] Concurrency Challenges in Database Systems: A Focus on Postgresql : веб-сайт. — URL: https://www.researchgate.net/publication/391365712_Concurrency_Challenges_in_Database_Systems_A_Focus_on_Postgresql (дата обращения: 05.10.2026).

[15] Prisma vs Drizzle: Rich ORM or Lightweight Builder? : веб-сайт. — URL: https://www.13labs.au/compare/prisma-vs-drizzle (дата обращения: 05.10.2026).

[16] Drizzle vs Prisma: Choosing the Right TypeScript ORM : веб-сайт. — URL: https://betterstack.com/community/guides/scaling-nodejs/drizzle-vs-prisma/ (дата обращения: 05.10.2026).

[17] Optimistic UI & Mutations : веб-сайт. — URL: https://fearchitect.com/topics/optimistic-ui-mutations (дата обращения: 05.10.2026).

[18] PostgreSQL Advisory Locks with pg_advisory_lock : веб-сайт. — URL: https://www.postgresscripts.com/post/postgresql-advisory-locks-with-pg-advisory-lock/ (дата обращения: 05.10.2026).

[19] How to Use Advisory Locks in PostgreSQL : веб-сайт. — URL: https://www.cybrosys.com/research-and-development/postgres/how-to-use-advisory-locks-in-postgresql (дата обращения: 05.10.2026).

[20] Solving concurrency issue with postgresql advisory locks : веб-сайт. — URL: https://www.mandricmihai.com/2021/01/solving-concurrency-issue-with-advisory-locks-in-python.html (дата обращения: 05.10.2026).

[21] Optimistic Updates | TanStack Query React Docs : веб-сайт. — URL: https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates (дата обращения: 05.10.2026).